

Chapter 1

Requirements, Impacts and Constraints

This chapter specifies the requirements for CAEM (Confidence-Aware Episodic Memory with Self-Improvement), documents the hardware constraints that directly shaped each design decision, analyses the system’s broader societal and ethical impact, and outlines the project management plan used to deliver the research within the available timeline and resources.

The requirements presented here follow directly from the three architectural problems identified in Chapter 1 — stateless operation, poor confidence calibration, and the absence of self-improvement — and from the eight processing stages described in the methodology overview. Every functional requirement corresponds to exactly one pipeline stage. Every non-functional requirement maps to a measurable threshold that determines whether the system has succeeded.

1.1 System Requirements

1.1.1 Functional Requirements

The CAEM pipeline processes a query through eight sequential stages. Each stage constitutes a functional requirement of the system. Table 1.1 summarises all eight requirements. The following paragraphs elaborate on their design intent.

FR-01 and **FR-02** impose a strict ordering constraint: pre-routing confidence must be estimated from the query alone, before any generation occurs. This is a deliberate design decision that avoids the cost of generating an answer only to discard it at the routing stage.

FR-03 introduces the OR-condition safety veto as a hard constraint separate from the routing score formula. A dangerously low u_{pre} signals that the model does not understand the query well enough to trust any memory match, regardless of how similar the retrieved episode appears. Combining these two mechanisms into a single formula would allow a high-quality memory match to arithmetically mask an untrustworthy model state — the OR-condition prevents this.

FR-05 establishes an asymmetric acceptance gate: the $\hat{u}$ threshold of 0.60 is set to preserve recall (avoid rejecting correct answers too early) while still escalating uncertain Tier-2 outputs to Tier 3. Stage 5 then computes the independent stored-confidence signal $\hat{u}_{\text{stored}}$ and applies the storage gate.

Table 1.1: CAEM functional requirements derived from the eight-stage pipeline architecture.

FR	Stage	Requirement
FR-01	Encode (Stage 1–2)	The system shall accept a natural-language query, tokenise it for the language model, and produce a 768-dimensional Sentence-BERT embedding (all-mpnet-base-v2) for memory retrieval.
FR-02	Pre-routing confidence (Stage 3a)	The system shall estimate a pre-routing confidence score $u_{\text{pre}} \in [0, 1]$ from token probability and an internal convergence signal C_{conv} , without performing full answer generation.
FR-03	Adaptive routing (Stage 3b)	The system shall route each query to exactly one of three processing tiers based on a routing score combining memory similarity and stored confidence, subject to an OR-condition safety veto that forces Tier 3 when $u_{\text{pre}} < 0.60$ regardless of memory similarity.
FR-04	Answer generation (Stages 4 and 6)	The system shall generate a full answer using the fine-tuned Flan-T5-Large model with a retrieved reasoning chain as a soft prompt (Tier 2), or using retrieval-augmented generation with Wikipedia passages as context (Tier 3).
FR-05	Post-generation gate (Stage 4a)	For Tier 2 answers only, the system shall estimate a post-generation confidence score $\hat{u} \in [0, 1]$ from four signals (token probability, MC Dropout, self-consistency, semantic entropy). If $\hat{u} < 0.60$, the query shall be escalated to Tier 3.
FR-06	Verification (Stage 5)	The system shall verify all non-Tier-1 answers through a three-signal pipeline: NLI entailment probability (p_{entail}) via RoBERTa-Large-MNLI, average pairwise self-consistency (s_{avg}) via Sentence-BERT, and normalised semantic entropy (h_{norm}), producing a stored confidence score $\hat{u}_{\text{stored}} = 0.50 p_{\text{entail}} + 0.30 s_{\text{avg}} + 0.20 (1 - h_{\text{norm}})$.
FR-07	Memory storage (Stage 7)	The system shall write a verified episode to the episodic memory store if and only if $\hat{u}_{\text{stored}} \geq 0.50$ and the query is sufficiently novel (nearest-neighbour cosine similarity < 0.95). Episodes shall be pruned by value score when the store exceeds 95% capacity.
FR-08	Self-improvement (Stage 8)	The system shall fine-tune the language model on verified reasoning chains after each cycle, using L2 regularisation ($\lambda = 0.01$) and a mixed training corpus (90% verified, 10% general-domain) to prevent catastrophic forgetting. Fine-tuning shall abort and revert to the previous cycle’s weights if the held-out general-

FR-08 introduces a forgetting circuit-breaker. Unlike naive continual learning approaches, which have been shown to produce 30–50% degradation of prior capabilities under repeated fine-tuning [?], CAEM monitors held-out general-domain retention after each cycle and reverts the model if the post/pre retention ratio falls below 0.93. MMLU is reported as an external retention check, but it is not the in-loop abort signal. This abort condition ensures that improvements on the target benchmarks never come at the cost of general language competence.

1.1.2 Non-Functional Requirements

Non-functional requirements constrain *how* the system meets its functional obligations. The targets in Table 1.2 are derived from three sources: the thesis’s computational budget, theoretical conditions of the data purity theorem, and the system’s intended deployment context.

Table 1.2: CAEM non-functional requirements with targets, sources, and measurement methods.

Requirement	Target	Source	Measurement
Tier 1 end-to-end latency	< 400 ms	Design constraint (retrieval only)	Wall-clock timing on RTX 3060
Tier 2 end-to-end latency	< 2.5 s	Design constraint	Wall-clock timing per call
Tier 3 end-to-end latency	< 6 s	Design constraint	Wall-clock timing per call
Verification balanced accuracy α	$\geq 85\%$	Purity theorem: $\alpha > 0.5$ required, $\alpha \geq 0.85$ ensures $P \gg p$	$(TP+TN)/N$ on validation set
General-domain retention ratio per cycle	$\geq 93\%$	Forgetting abort threshold (FR-08)	Held-out general-domain accuracy ratio (post/pre cycle)
GPU budget (total)	≤ 50 h	RTX 3060 university allocation	Cumulative GPU time across all cycles
Memory store capacity	$\leq 20,000$ episodes	VRAM and disk budget	Episode count before pruning triggers
Embedding dimension	768 dimensions	<code>all-mpnet-base-v2</code> specification	Verified at initialisation

The latency targets define the three tiers as a performance spectrum rather than an arbitrary categorisation. Tier 1 must be fast enough to represent a meaningful efficiency gain over generation; 400 ms achieves this for interactive use. Tier 3 at 6 s reflects the genuine cost of Wikipedia retrieval and full generation; the system earns this latency only for queries that cannot be served from memory.

The verification accuracy target of $\alpha \geq 85\%$ is not arbitrary. The data purity theorem (Chapter 4, Theorem 1) proves that memory purity $P > p$ (base accuracy) if and only if $\alpha > 0.5$. The 85% target provides substantial margin above this theoretical minimum, ensuring that even under realistic calibration error, memory quality remains well above base generation quality across all four benchmarks.

The general-domain retention target of 93% per cycle is tied directly to FR-08’s circuit-breaker. This threshold was calibrated conservatively: preliminary implementation tests showed that L2-regularised fine-tuning with $\lambda = 0.01$ achieves $1.75\times$ above-baseline retention in Cycle 1, comfortably clearing the 93% floor. The abort condition makes this a hard safety constraint rather than a best-effort target.

1.1.3 Hardware Constraints as Design Drivers

The hardware available for this research — an NVIDIA RTX 3060 with 12 GB VRAM and a 50 GPU-hour university allocation — is not merely a limitation to work around. Every major architectural decision in CAEM was shaped by these constraints, and the thesis’s central contribution is demonstrating that meaningful, measurable hallucination reduction is achievable within real-world resource limits, not in spite of them.

Table 1.3 documents each constraint and the design decision it forced.

Table 1.3: How each hardware constraint produced a specific CAEM design decision.

Constraint	Limit	Design decision forced
VRAM capacity	12 GB	Training batch size fixed at 4 (not 16). Previous-cycle weights (θ_{prev}) stored on CPU and transferred per batch to avoid holding two full model copies in VRAM simultaneously.
Passage index size	500K Wikipedia passages	Limits Tier 3 retrieval quality relative to full-scale DPR deployments, but remains within the 10 GB FAISS index that fits on disk alongside the model.
GPU budget	50 h total	Drives $n_{\text{questions}} = 5,000$ per benchmark (not 10,000+) and $n_{\text{cycles}} = 3$ (not 5–7). Three cycles capture the essential self-improvement dynamics while using approximately 40–45 GPU hours.
Memory store capacity	$\leq 20,000$ episodes	Drives the cold-start seeding strategy (447 episodes seeded before Cycle 0 to prevent empty-memory cold-start degradation) and the value-based pruning mechanism that activates at 95% capacity.

The RTX 3060’s 12 GB VRAM also prevents loading the NLI model (RoBERTa-Large-MNLI, 355M parameters) and the main model (Flan-T5-Large, 780M parameters) simultaneously at full precision. The solution — loading both in `fp16` and reserving gradient storage only during fine-tuning — was implemented as a design constraint at the system level rather than a runtime workaround.

These constraints are a research contribution in themselves. Much of the existing self-improvement literature assumes access to multi-GPU clusters or 70B-scale models. CAEM demonstrates a complete verified self-improvement loop under single-GPU, 12 GB constraints, making the approach reproducible at institutions without dedicated AI infrastructure. If the full experiment is later rerun on a higher-capacity device (e.g. RTX 4090 or A100), Chapter 5 will report latency numbers from that device; the architectural decisions above remain unchanged regardless.

With the functional, non-functional, and hardware requirements established, the next section turns to the broader question of impact: the societal value the system can deliver, the risks it introduces, and the boundaries within which its claims hold.

1.2 Societal and Ethical Impact

1.2.1 Potential Benefits and Societal Value

The primary societal benefit of CAEM is reducing the rate at which factually incorrect information is generated with high confidence by language model systems. Hallucination in deployed AI has measurable real-world costs: medical literature analysis has found hallucination rates of 69–77% in ChatGPT responses to clinical questions [?], and the TruthfulQA benchmark demonstrates that current frontier models answer only 58% of questions truthfully alone and as low as 21–25% on the combined truthful-and-informative metric, a performance substantially below human accuracy [?].

CAEM addresses this through three complementary mechanisms. First, it prevents unverified knowledge from accumulating in memory: only answers that pass multi-signal verification are stored, so the system’s knowledge base cannot grow through confabulation. Second, it makes accumulated knowledge reusable: verified reasoning chains become training data that improves future generation, avoiding the stateless wastage of current systems in which each correct answer is discarded after delivery. Third, it provides a principled confidence signal that separates high-reliability Tier 1 responses from uncertain Tier 3 queries, allowing downstream systems or users to act on this signal appropriately.

These properties make the system particularly relevant to knowledge-intensive applications where hallucination has high costs: educational content delivery, enterprise knowledge management, and domain-specific question answering. The verified episodic memory design also enables an interpretable audit trail — every stored episode records its source query, answer, verification scores (p_{entail} , s_{avg} , h_{norm}), and cycle provenance, providing a level of transparency absent from purely generative systems.

1.2.2 Risks and Ethical Considerations

CAEM introduces several risks that must be acknowledged alongside its benefits.

False positive storage. The multi-signal verification pipeline targets $\alpha \geq 85\%$ balanced accuracy, which means approximately 10–15% of stored episodes may contain incorrect answers that passed the verification gate. An incorrect episode stored with high

$\hat{u}_{\text{stored}}$ will be retrieved for similar future queries and may also appear as training data in subsequent fine-tuning cycles, potentially reinforcing the error. This is the primary systemic risk of the design. Mitigation mechanisms — retroactive re-verification between cycles and the $\hat{u}_{\text{stored}} \geq 0.50$ pruning threshold — reduce but cannot eliminate this risk. Chapter 5 quantifies the observed false positive rate empirically.

Miscalibrated confidence. If temperature scaling calibration is performed incorrectly or on an unrepresentative calibration set, the $\hat{u}$ thresholds may not correspond to actual accuracy rates. A miscalibrated system could route low-quality answers through Tier 1 (direct retrieval) without triggering the verification gate, compounding errors silently. Calibration is therefore performed on a held-out 500-sample set, and the expected calibration error (ECE) is reported in Chapter 5 to document calibration quality.

Benchmark scope. The system is evaluated exclusively on English-language, closed-form question answering tasks with objectively verifiable answers. Claims about hallucination reduction are scoped to this setting. The system should *not* be interpreted as a general solution to hallucination in open-ended generation, medical advice, legal reasoning, or multilingual contexts. These domains introduce evaluation criteria — subjectivity, jurisdiction-specific facts, code-switching — that the current design does not address.

Model selection and bias. Flan-T5-Large inherits biases present in its pre-training corpus [?]. Fine-tuning on verified episodic memory does not correct for distributional biases in the base model; it can only improve factual correctness within the knowledge distribution already captured during pre-training. Users and downstream integrators should be aware that high $\hat{u}_{\text{stored}}$ indicates that an answer is self-consistent and entailed by the model’s own reasoning, not that it is verified against a trusted external ground truth.

1.2.3 Scope and Deployment Limitations

This research is an undergraduate thesis project and its claims are correspondingly scoped. The experimental results apply to the specific combination of model (Flan-T5-Large, 780M parameters), benchmarks (HotpotQA, TruthfulQA, FEVER, StrategyQA), and hardware (RTX 3060, 12 GB VRAM). Generalisation to larger models, real-time deployment environments, or domains outside knowledge-intensive QA requires further investigation.

The closed-loop design does not involve any human feedback signal, user data collection, or external API calls during inference. Memory episodes are derived entirely from model-generated content, and all training data is either pre-existing (Flan-T5’s original training) or generated by the model itself. No personally identifiable information is collected or processed.

With the impact and risk analysis complete, the final section describes how the research is managed to deliver these components within the available time and resource constraints.

1.3 Project Management

1.3.1 Phases and Timeline

The research is structured into four sequential phases, each with defined deliverables and gate conditions that must pass before the subsequent phase begins. Figure 1.1 gives a visual overview of the four-phase progression; Table 1.4 details the deliverables for each phase.

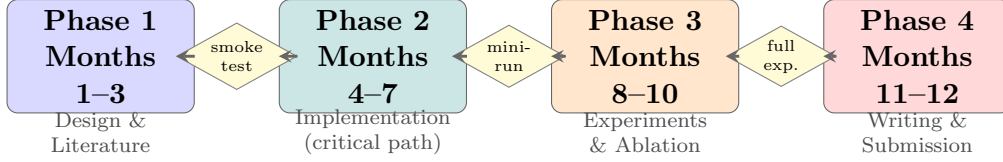


Figure 1.1: Four-phase project timeline. Each diamond represents a gate condition (smoke test, mini-run validation, and full-scale experiment completion respectively) that must pass before the next phase begins. Phase 2 is the critical path: the full-scale experiment cannot start until all eight pipeline stages run end-to-end without failures.

The twelve-month allocation across phases and the deliverables for each gate are described in Table 1.4.

Table 1.4: Phase-based project plan with deliverables and dependencies.

Ph.	Duration	Period	Deliverables and Gate Conditions
1	3 months	Months 1–3	Literature review (Ch. 2). System design: pipeline architecture, confidence estimation design, routing logic, verification protocol. Pre-thesis report submitted with Chapters 1–2 and methodology overview.
2	4 months	Months 4–7	Full implementation of all 8 pipeline stages. Passage index construction (500K Wikipedia passages). Cold-start seeding (447 episodes). Smoke test on RTX 3060: Cycle 0–3 on $n = 8$ synthetic queries, all stages verified end-to-end.
3	3 months	Months 8–10	Mini-run ($n = 500$ per benchmark, 4 cycles) to validate improvement signal and mechanism evidence. Full-scale experiment ($n = 5,000$ per benchmark, 3 cycles) on confirmed device. Ablation study and purity validation scripts.
4	2 months	Months 11–12	Chapter writing (Ch. 3–6). Final corrections. Thesis submission.

Phase 2 is the critical path. The full-scale experiment (Phase 3) cannot begin until all eight pipeline stages pass the smoke test with no exception-raising failures. Chapter 5 similarly cannot be written until Phase 3 is complete.

1.3.2 Risk Register

Table 1.5 identifies the four highest-impact risks to the project timeline and documents the contingency action for each.

Table 1.5: Key project risks, likelihood, and mitigation actions.

Risk	Likelihood	Impact	Mitigation
GPU OOM during fine-tuning	Medium	High	Batch size = 4. θ_{prev} on CPU. Gradient checkpointing. Dynamic padding via <code>DataCollatorForSeq2Seq</code> .
Verification $\alpha < 85\%$ (below NFR-04)	Low	Medium	Increase NLI sample count M . Retune $\hat{u}_{\text{stored}}$ threshold. Report boundary condition as a finding rather than a failure.
General-domain retention ratio below 93% abort threshold	Low	Medium	Increase general-domain mixing ratio from 10% to 20%. Reduce learning rate from 10^{-5} to 5×10^{-6} . Revert to previous cycle’s model weights.
Full-scale GPU device unavailable in Phase 3	Medium	Medium	Mini-run ($n = 500$) on RTX 3060 provides publishable results. Scale to $n = 5,000$ on Kaggle T4, Vast.ai, or university cluster if available before submission.

1.3.3 Development Methodology

The implementation follows a test-driven development approach. Each pipeline stage is implemented with an accompanying unit test suite that verifies both the happy path and common failure modes (e.g. empty memory store, NLI model unavailable, FAISS index not built). At the time of writing, 96 unit tests and 3 prompt sanity tests pass with no failures across the complete pipeline codebase.

Configuration management uses a centralised `CAEMConfig` dataclass (`caem/config.py`) from which all numeric constants are drawn. No magic numbers appear in pipeline code. Every hyperparameter is annotated with its category (literature-fixed, design choice, or empirically calibrated) and its source, enabling full traceability from thesis claim to implementation constant.

Version control and reproducibility are maintained throughout. The implementation log (`caem-implementation-log.md`) records every resolved bug, design revision, and experimental result chronologically, providing a complete record of decisions made between Chapters 2 and 5.

This chapter has established the complete specification boundary for CAEM. Section 1.1 grounded eight functional requirements in the pipeline stages and bounded performance through measurable non-functional targets; the hardware subsection showed that real resource constraints drove specific architectural choices rather than limiting the design arbitrarily. Section 1.2 framed the societal value of the work and documented the risks — particularly false positive storage and miscalibrated confidence — that experimental evaluation in Chapter 5 must address. Section 1.3 defined the gated four-phase delivery plan and the test-driven development practice that keeps implementation and specifi-

cation aligned throughout. Chapter 4 now translates these requirements into the full technical methodology: the mathematical formalisation of each pipeline stage, the confidence estimation equations, the verification protocol, and the theoretical analysis of the self-improvement dynamics.